C++ ARRAYS

A Complete Beginner-Friendly Guide

From one-dimensional arrays to two-dimensional arrays, strings, searching, sorting, functions, and practical problem solving.

Learning goal: By the end of this guide, a beginner should be able to declare, initialize, access, update, loop through, search, and manipulate arrays in C++.

1. What Is an Array?

An array is a collection of values of the same data type stored under one variable name. Instead of creating separate variables such as mark1, mark2, mark3, and mark4, an array lets us store many related values together.

```
int marks[5];
```

The number 5 tells C++ that the array has five positions. For a normal C++ array, positions are indexed from 0, so the valid indexes are 0, 1, 2, 3, and 4.

Important beginner rule: the number of elements is not the same thing as the last index. An array of 5 elements ends at index 4.

Example 1: Five marks

Stores five integer marks.

```
int marks[5] = {70, 65, 80, 90, 55};
```

Example 2: Five ages

Stores five ages.

```
int ages[5] = {18, 19, 20, 21, 22};
```

Example 3: Five prices

Stores five decimal prices.

```
double prices[5] = {10.5, 20.0, 7.5, 15.2, 30.0};
```

Example 4: Five names (characters)

A character array can hold characters.

```
char grade[5] = {'A', 'B', 'C', 'D', 'E'};
```

Example 5: Five temperatures

Stores five temperature readings.

```
float temperature[5] = {25.5, 26.0, 24.8, 27.1, 23.9};
```

2. Declaring Arrays

Declaration means telling C++ the array's data type, name, and size. The general form is: **dataType arrayName[size];**.

Example 1: Integer array

Creates 10 integer positions.

```
int numbers[10];
```

Example 2: Double array

Creates 4 positions for decimal values.

```
double prices[4];
```

Example 3: Float array

Creates 6 positions for float values.

```
float rainfall[6];
```

Example 4: Character array

Creates 8 character positions.

```
char letters[8];
```

Example 5: Boolean array

Creates 3 true/false positions.

```
bool passed[3];
```

3. Initializing Arrays

Initialization gives the array its starting values. You can provide all values, or let C++ determine the size when values are supplied.

Example 1: Initialize all values

Five values fill five positions.

```
int marks[5] = {70, 80, 65, 90, 75};
```

Example 2: Let C++ count

The compiler counts the four values.

```
int numbers[] = {10, 20, 30, 40};
```

Example 3: Partial initialization

Remaining elements are initialized to zero.

```
int values[5] = {10, 20};
```

Example 4: All zero

An empty initializer sets all elements to zero.

```
int scores[5] = {};
```

Example 5: Character values

Characters can be initialized individually.

```
char vowels[5] = {'a', 'e', 'i', 'o', 'u'};
```

4. Accessing Array Elements

Use the index inside square brackets. The first element is index 0.

```
int marks[3] = {50, 60, 70};
cout << marks[0];  // prints 50
cout << marks[2];  // prints 70
```

Example 1: First element

Access index 0.

```
cout << marks[0];
```

Example 2: Third element

Access index 2.

```
cout << marks[2];
```

Example 3: Store an element

Copy the second element into x.

```
int x = marks[1];
```

Example 4: Use in calculation

Add two elements.

```
int total = marks[0] + marks[1];
```

Example 5: Update an element

Change the fourth position.

```
marks[3] = 100;
```

5. Updating Array Elements

An array element can be changed by assigning a new value to its index.

Example 1: Change one mark

Replace the value at index 1.

```
marks[1] = 95;
```

Example 2: Change first value

Replace index 0.

```
numbers[0] = 500;
```

Example 3: Increase a value

Add 5 to an existing element.

```
marks[2] = marks[2] + 5;
```

Example 4: Multiply a value

Double one element.

```
prices[3] = prices[3] * 2;
```

Example 5: Reset a value

Set an element to zero.

```
scores[4] = 0;
```

6. Traversing Arrays with for Loops

Traversal means visiting each array element, usually from the first element to the last. A for loop is one of the most useful tools for this.

```
int marks[5] = {60, 70, 80, 90, 100};

for (int i = 0; i < 5; i++) {
    cout << marks[i] << endl;
}
```

Notice the condition **i < 5**. It stops before 5 because index 5 does not exist in a five-element array.

Example 1: Print all elements

Print every value on one line.

```
for (int i=0; i<5; i++) cout << marks[i] << " ";
```

Example 2: Print with line breaks

Print each value on a separate line.

```
for (int i=0; i<5; i++) cout << marks[i] << endl;
```

Example 3: Add all values

Use a running total.

```
int total=0; for(int i=0;i<5;i++) total += marks[i];
```

Example 4: Double values

Update every element.

```
for(int i=0;i<5;i++) marks[i] *= 2;
```

Example 5: Read values from user

Store five keyboard inputs.

```
for(int i=0;i<5;i++) cin >> marks[i];
```

7. Finding Array Size

For a built-in array in the same scope where it is an actual array, **sizeof(array) / sizeof(array[0])** can calculate the number of elements.

Example 1: Five integers

Each int has the same size, so the ratio gives element count.

```
int a[5] = {1,2,3,4,5};\nint n = sizeof(a) / sizeof(a[0]);
```

Example 2: Ten doubles

Works for other element types too.

```
double a[10];\nint n = sizeof(a) / sizeof(a[0]);
```

Example 3: Characters

Find character-array element count.

```
char a[8];\nint n = sizeof(a) / sizeof(a[0]);
```

Example 4: Use in loop

Avoid hard-coding the size.

```
for (int i=0; i<sizeof(a)/sizeof(a[0]); i++) cout << a[i];
```

Example 5: Print size

Display the calculated number.

```
cout << sizeof(a)/sizeof(a[0]);
```

8. Input into Arrays

A loop is normally used to ask the user for several values.

Example 1: Enter marks

Read five marks.

```
int marks[5];\nfor(int i=0;i<5;i++) cin >> marks[i];
```

Example 2: Enter ages

Read five ages.

```
int ages[5];\nfor(int i=0;i<5;i++) cin >> ages[i];
```

Example 3: Enter prices

Read decimal prices.

```
double prices[5];\nfor(int i=0;i<5;i++) cin >> prices[i];
```

Example 4: Enter temperatures

Read float readings.

```
float t[7];\nfor(int i=0;i<7;i++) cin >> t[i];
```

Example 5: Display after input

Read then display values.

```
for(int i=0;i<3;i++) cin >> a[i];\nfor(int i=0;i<3;i++) cout << a[i] << " ";
```

9. Sum, Average, Maximum and Minimum

Arrays are often used for data analysis. A loop can calculate totals and identify the largest or smallest value.

Example 1: Sum

Add every element.

```
int sum=0; for(int i=0;i<5;i++) sum += marks[i];
```

Example 2: Average

Divide the sum by the number of elements.

```
double avg = (double)sum / 5;
```

Example 3: Maximum

Keep the largest value found so far.

```
int max=marks[0]; for(int i=1;i<5;i++) if(marks[i]>max) max=marks[i];
```

Example 4: Minimum

Keep the smallest value found so far.

```
int min=marks[0]; for(int i=1;i<5;i++) if(marks[i]<min) min=marks[i];
```

Example 5: Count passes

Count values at least 50.

```
int count=0; for(int i=0;i<5;i++) if(marks[i]>=50) count++;
```

10. One-Dimensional Arrays (1D)

A one-dimensional array is a simple list. It has one index.

Example 1: Student marks

A list of marks.

```
int marks[5] = {70,80,60,90,75};
```

Example 2: Monthly sales

A list of sales amounts.

```
double sales[4] = {1200,1500,900,1800};
```

Example 3: Employee ages

A list of ages.

```
int ages[4] = {25,31,28,40};
```

Example 4: Product quantities

A list of quantities.

```
int qty[6] = {3,5,2,8,1,4};
```

Example 5: Daily rainfall

A list of seven readings.

```
double rain[7] = {2.1,0,4.5,3.2,1.0,0.5,2.8};
```

11. Two-Dimensional Arrays (2D)

A two-dimensional array is like a table made of rows and columns. Its syntax is **array[row][column]**.

```
int marks[3][2] = {
    {2, 7},
    {10, 30},
    {67, 88}
};
```

This contains 3 rows and 2 columns, so it stores 6 integers. Indexing still begins at zero.

Example 1: Class marks

Three students, two subjects.

```
int marks[3][2] = {{70,80},{65,75},{90,85}};
```

Example 2: Matrix

A 2 by 2 matrix.

```
int matrix[2][2] = {{1,2},{3,4}};
```

Example 3: Sales table

Three products, two months.

```
double sales[3][2] = {{10,20},{30,40},{50,60}};
```

Example 4: Temperatures

Two cities, three days.

```
float temp[2][3] = {{25,26,27},{22,24,23}};
```

Example 5: Seats

Three rows, four seats.

```
bool seats[3][4] = {{true,false,true,true},{false,true,true,false},{true,true,false,false}};
```

12. Nested Loops for 2D Arrays

A nested loop means one loop inside another. The outer loop normally moves through rows and the inner loop moves through columns.

```
for (int row=0; row<3; row++) {
    for (int col=0; col<2; col++) {
        cout << marks[row][col] << " ";
    }
    cout << endl;
}
```

Example 1: Print a 3x2 table

Outer loop handles rows; inner loop handles columns.

```
for(int r=0;r<3;r++){ for(int c=0;c<2;c++) cout<<a[r][c]<<" "; cout<<endl; }
```

Example 2: Sum a 2x2 matrix

Visit all four cells.

```
int sum=0; for(int r=0;r<2;r++) for(int c=0;c<2;c++) sum+=a[r][c];
```

Example 3: Input a table

Read each row and column.

```
for(int r=0;r<3;r++) for(int c=0;c<4;c++) cin>>a[r][c];
```

Example 4: Display with headings

Use the outer loop to create rows.

```
for(int r=0;r<2;r++){ for(int c=0;c<3;c++) cout<<a[r][c]<<" "; cout<<"\n"; }
```

Example 5: Find a value

Check every cell.

```
for(int r=0;r<3;r++) for(int c=0;c<3;c++) if(a[r][c]==50) cout<<"Found";
```

13. Character Arrays and C-Style Strings

A character array stores characters. A C-style string normally ends with the null character **\0**. Modern C++ also provides std::string, which is generally easier and safer for text.

Example 1: Character list

Store individual characters.

```
char letters[4] = {'A','B','C','D'};
```

Example 2: C-style string

The literal automatically includes a null terminator.

```
char name[] = "Mwamba";
```

Example 3: Print a character

Access one character.

```
cout << name[0];
```

Example 4: Change a character

Replace one character.

```
name[0] = 'm';
```

Example 5: Use std::string

Preferred for most beginner text programs.

```
#include <string>\nstring name = "Mwamba";
```

14. Searching Arrays

Linear search checks elements one by one until it finds the target or reaches the end.

Example 1: Find a mark

Compare every mark with target.

```
int target=80; for(int i=0;i<5;i++) if(marks[i]==target) cout<<"Found";
```

Example 2: Record index

Save where the value was found.

```
int pos=-1; for(int i=0;i<5;i++) if(a[i]==target){pos=i; break;}
```

Example 3: Count occurrences

Count how many times target appears.

```
int count=0; for(int i=0;i<5;i++) if(a[i]==target) count++;
```

Example 4: Search a name

For C-style strings, compare using string functions or use std::string.

```
string names[3]={"A","B","C"};\nif(names[1]=="B") cout<<"Found";
```

Example 5: Stop early

break ends the loop after finding the first match.

```
for(int i=0;i<5;i++){ if(a[i]==50){ cout<<"Found"; break; } }
```

15. Sorting Arrays

Sorting means arranging values, for example from smallest to largest. A simple beginner method is bubble sort.

```
for (int i=0; i<4; i++) {
    for (int j=0; j<4-i; j++) {
        if (a[j] > a[j+1]) {
            int temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}
```

Example 1: Ascending order

Smallest to largest using bubble sort.

```
if(a[j] > a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
```

Example 2: Descending order

Largest to smallest by reversing the comparison.

```
if(a[j] < a[j+1]) { int t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
```

Example 3: Sort five values

Use two loops to compare neighbors.

```
for(int i=0;i<4;i++) for(int j=0;j<4-i;j++) if(a[j]>a[j+1]) swap(a[j],a[j+1]);
```

Example 4: Use std::sort

C++ provides a library sorting function.

```
#include <algorithm>\nsort(a, a+5);
```

Example 5: Sort then search

Sorted data can be searched efficiently with suitable algorithms.

```
sort(a,a+5);\n// then perform a search
```

16. Arrays and Functions

Arrays can be passed to functions so that a function can process a collection of values. A common beginner form passes the array together with its size.

Example 1: Print array

Function receives array and size.

```
void print(int a[], int n) { for(int i=0;i<n;i++) cout<<a[i]<<" "; }
```

Example 2: Sum array

Return the total.

```
int sum(int a[], int n) { int s=0; for(int i=0;i<n;i++) s+=a[i]; return s; }
```

Example 3: Find maximum

Return largest element.

```
int maxVal(int a[], int n) { int m=a[0]; for(int i=1;i<n;i++) if(a[i]>m)m=a[i]; return m; }
```

Example 4: Change values

Function can modify elements.

```
void doubleValues(int a[], int n) { for(int i=0;i<n;i++) a[i]*=2; }
```

Example 5: Use function

Call a function with array and size.

```
int a[5]={1,2,3,4,5};\nprint(a,5);
```

17. Common Array Errors

Example 1: Wrong last index

A five-element array ends at index 4, not 5.

```
int a[5];\na[5] = 100; // ERROR: out of bounds
```

Example 2: Off-by-one loop

Use i < size, not i <= size.

```
for(int i=0;i<5;i++) cout<<a[i];
```

Example 3: Uninitialized values

Do not assume local arrays contain useful values.

```
int a[5]; // values are indeterminate
```

Example 4: Wrong 2D dimensions

Respect both row and column limits.

```
int a[3][2];\na[2][1] = 88; // valid
```

Example 5: Type mismatch

Store compatible values in the declared type.

```
int a[3];\n// Prefer integer values for this array
```

18. Complete Practical Program

This example combines declaration, input, traversal, sum, average, maximum, and minimum.

```
#include <iostream>
using namespace std;

int main() {
    int marks[5];
    int sum = 0;

    for (int i=0; i<5; i++) {
        cout << "Enter mark " << i+1 << ": ";
        cin >> marks[i];
        sum += marks[i];
    }

    int maximum = marks[0];
    int minimum = marks[0];

    for (int i=1; i<5; i++) {
        if (marks[i] > maximum) maximum = marks[i];
        if (marks[i] < minimum) minimum = marks[i];
    }

    double average = (double)sum / 5;

    cout << "Average: " << average << endl;
    cout << "Highest: " << maximum << endl;
    cout << "Lowest: " << minimum << endl;

    return 0;
}
```

The program first creates an array for five marks. The first loop collects input and adds each mark to sum. The second loop compares values to find the highest and lowest. Finally, the program calculates the average and prints the results.

19. Quick Rules to Remember

• Array indexes normally start at 0.

• If an array has N elements, valid indexes are 0 through N-1.

- All elements of a built-in array have the same data type.
- Use a loop to process many elements efficiently.
- For 2D arrays, use [row][column].
- Nested loops are commonly used for 2D arrays.
- Never intentionally access an index outside the array.
- Initialize arrays when practical, especially before reading or calculating.
- For text, std::string is usually easier than raw character arrays.
- Always know the array size when writing loops or pass the size to functions.

20. PRACTICAL SCENARIO QUESTIONS — 25 QUESTIONS

Try to write a complete C++ program for each question. Do not only write the answer. Declare the array, process it with appropriate loops, and display meaningful output. The questions progress from beginner to intermediate.

1. Student Marks

Create an array for 10 student marks. Ask the user to enter all marks and display them.

2. Total Marks

Using an array of 5 marks, calculate and display the total.

3. Average Mark

Store 8 marks and calculate the average.

4. Highest Mark

Store 10 marks and find the highest mark.

5. Lowest Mark

Store 10 marks and find the lowest mark.

6. Pass and Fail

Store 10 marks. Count how many students passed with 50 or more and how many failed.

7. Even and Odd

Store 10 integers and count how many are even and how many are odd.

8. Positive and Negative

Store 10 integers and count positive, negative, and zero values.

9. Search a Number

Store 10 integers. Ask for a target and report whether it exists and its index if found.

10. Count Occurrences

Store 10 integers. Ask for a target and count how many times it occurs.

11. Reverse an Array

Store 5 integers and display them in reverse order without changing the original array.

12. Copy an Array

Store 5 values in one array and copy them into a second array.

13. Largest and Smallest Together

Store 10 numbers and display both the maximum and minimum.

14. Monthly Sales

Store sales for 12 months and calculate annual sales and average monthly sales.

15. Temperature Analysis

Store 7 daily temperatures and find the hottest and coldest day.

16. Product Quantities

Store quantities for 8 products and calculate the total quantity.

17. Price Filter

Store 10 product prices and display only prices greater than 25.

18. Array Sorting

Store 8 numbers and sort them from smallest to largest.

19. Descending Sort

Store 8 numbers and sort them from largest to smallest.

20. Student Subject Table

Create a 2D array for 5 students and 3 subjects. Display all marks.

21. Subject Totals

Using the same 2D array, calculate the total mark for each student.

22. Class Subject Average

Using a 2D marks array, calculate the average for each subject.

23. Cinema Seats

Create a 2D array representing 5 rows and 6 cinema seats. Use true for occupied and false for available. Count available seats.

24. Shop Sales Table

Create a 2D array for 4 products and 3 months. Calculate total sales for each product.

25. Mini Student Report

Create a program using an array that inputs 10 marks, displays all marks, calculates total and average, finds highest and lowest, counts passes/fails, and searches for a mark entered by the user.

21. Suggested Challenge Order

For the best learning progression, first complete Questions 1–10 without looking at solutions. Then attempt 11–18. Finally, use nested loops for Questions 20–25. If you get stuck, break each problem into four steps: (1) declare the array, (2) input or initialize data, (3) process it with a loop, and (4) display the result.